`Mescaline-1.0`: Users guide

Hayley J. Macpherson

October 25, 2023

`Mescaline` is an analysis code written in Fortran by H. J. Macpherson and D. J. Price for post-processing analysis of Einstein Toolkit (ET) data – specifically intended for inhomogeneous cosmology. It calculates various tensors, vectors, and scalars in a general inhomogeneous spacetime as output from a numerical relativity simulation. See Macpherson et al. (2018, 2019) for some applications of the code.

In this document we outline the basics of getting set up with certain dependencies, compiling, and using `mescaline`. We also give details on what is actually calculated, including equations, methods, explanations, etc., of the way certain things are implemented. We also include some consistency tests at the end using analytic metrics.

The information in this users guide is for the first publicly-available version of `mescaline`. This version is based on calculating spatial averages on your simulation hypersurfaces, specifically intended for the analysis of backreaction effects. The calculations included with this version can be pushed beyond this application, with 3D output of many interesting space-time tensors included if you wish. Please explore.

Contents

1 Usage

`mescaline` reads in HDF5 file format as (almost) written by the Einstein Toolkit (ET). Make sure your simulation has the following output: $\gamma_{ij}, K_{ij}, \alpha, \rho, v^i$. `Mescaline` is based on `ADMBase` physical spatial metric (γ_{ij}), extrinsic curvature (K_{ij}), and lapse function (α); and `HydroBase` rest-mass density (ρ) and 3-velocity (v^i). See the documentation of these thorns for specific definitions. We always assume zero shift vector.

If your ET simulation has written 3-D data in "variable-chunked" format (i.e., one file per variable which contains every time step), you must first split these files into "iteration-chunked" format (one file for each iteration containing every variable) for use with `mescaline`. In `mescaline/tools/` directory we include a Python script `split_HDF5_per_iteration3.py` (for Python version 3 and above) to split these files into a more manageable format. This script will split your ET output HDF5 data (e.g., `rho.xyz.h5`) into iteration-chunked data. The format resulting from this script is the format that `mescaline` can read.

First, run this script from your simulation directory. Note this might take a while for larger simulations, however, it only needs to be done once.

See the README for simple instructions to get started. We elaborate on some of these details here.

1.1 Dependencies

1.1.1 SPLASH

SPLASH is a visualisation tool for SPH simulations, written by Daniel Price. You first must download the most recent version of SPLASH, since `mescaline` borrows some of its modules for reading HDF5 data. See here for download instructions, or, just use Homebrew:

```
brew tap danieljprice/all

brew install splash
```

Once that is done, make sure you have set the environment variable `SPLASH_DIR` to the location of your `splash/` directory, e.g.

`export SPLASH_DIR= /splash`.

Note that SPLASH can also be used to visualise ET output data (using the split files) by compiling (after a normal compile) with `make SYSTEM=gfortran cactus`, which will give you an executable `csplash-hdf5` which can then be used to read Cactus HDF5 data. **You do not need to compile SPLASH to use `mescaline`. You just need to point to the files by setting `SPLASH_DIR`.**

1.1.2 HDF5

You need to have HDF5 installed to read (and write) HDF5 files. You will need to set the environment variable `HDF5ROOT` to the location of your HDF5 root directory, as this is used during compilation of `mescaline`.

1.1.3 C and Fortran compilers

`mescaline` has been tested to compile using both GNU and Intel compilers (though different versions may behave differently). Set the environment variable `SYSTEM=gfortran` or `SYSTEM=ifort` before compiling to choose GNU or Intel compilers, respectively.

1.2 Code structure

The `mescaline src/` directory is split into different branches of `mescaline`. In version 1.0, only the spatial averaging branch is available, the code for which lives in `src/avgs/`. Another directory, `src/tyrosine/`, contains the "core" modules which are used by every branch of `mescaline`, these should remain unchanged with future versions.

1.3 Compiling

Each `src/` directory contains an options file (named `options_<dirname>.f90`) which contains all parameter settings required for using that part of the code. Ensure you have set **all** of these options to values consistent with your simulation and desired analysis run **before** compiling. You must "make clean" and then "make" after any changes to this file.

To compile, type `make`. This will generate an executable named `bin/mescaline`.

1.4 Running

To run `mescaline` first make sure that the options you have set align with the simulation data you are presenting. If not, you could get weird results.

You can either give `mescaline` a long list of files, or a single file (again, so long as the options are consistent). You **should** get various errors for most of the inconsistencies I've come across – if you find one that you think needs a flag, please let me know!

The general usage is simple, e.g.

```
mescaline *.hdf5
```

2 Assumptions

There are a few simple assumptions in `mescaline`, as follows:

1. geometric units with $G = c = 1$
2. a uniform cubic computational grid with dx = dy = dz and hence nx = ny = nz
3. periodic boundary conditions implemented when taking all derivatives

4. gauge enforced such that the shift vector $\beta^i = 0$, with $\partial_t \beta^i = 0$

5. the evolution of the lapse is left general in all calculations (with choices in `options.f90`), and a routine specifying the form of $\partial_t \alpha$ can be changed to whatever you like in `get_dtalp` in `analytic_solns.f90`

3 Technical details

3.1 Definition and notations

We use Greek letters to denote spacetime indices which take values $0 \ldots 3$, and Latin indices for spatial indices which take values $1 \ldots 3$, and summation over repeated indices is assumed. ∇_α is the covariant derivative associated with the metric $g_{\mu\nu}$, and D_i is the spatial covariant derivative associated with the spatial metric γ_{ij}. See Macpherson (2019) for a breakdown of the $3+1$ foliation of spacetime, as notation should be relatively consistent here. $\partial_\alpha \equiv \partial/\partial x^\alpha$ is the partial derivative with respect to coordinate $x^\alpha = (ct, x^i)$.

3.2 Averaging in general foliations

Buchert et al. (2018, 2020) describe their new mass-preserving, fluid-intrinsic averaging formalism for general spacetime foliations. Here we outline some specifics related to how this is implemented in `mescaline`.

See Macpherson (2019) for some details of calculating the spatial Ricci tensor, scalar, trace of the extrinsic curvature, Christoffel symbols. These might be included here later, but they are just spatial derivatives of the metric, so maybe this is self-explanatory.

3.2.1 Expansion scalar

The expansion scalar Θ is defined by

$$\Theta \equiv \nabla_\alpha u^\alpha, \tag{1}$$

where ∇_α is the covariant derivative associated with the metric, and u^α is the four-velocity of the fluid. This becomes

$$\Theta = \partial_\alpha u^\alpha + {}^{(4)}\Gamma^\alpha_{\alpha\beta} u^\beta, \tag{2}$$

where ${}^{(4)}\Gamma^\alpha_{\alpha\beta}$ are the four-dimensional Christoffel symbols. Expanding these sums we find

$$\Theta = \partial_0 u^0 + \partial_i u^i + \frac{u^0}{\alpha}\partial_0(\alpha) - \alpha K u^0 + \frac{1}{\alpha}\partial_i(\alpha) u^i + \Gamma^j_{ji} u^i, \tag{3}$$

where we can write

$$\partial_0 u^0 = -\frac{1}{\alpha^2}\partial_0 u_0 + \frac{2u_0}{\alpha^3}\partial_0 \alpha, \tag{4}$$

and therefore

$$\Theta = \partial_i u^i - \frac{1}{\alpha^2}\partial_0 u_0 - \frac{u^0}{\alpha}\partial_0(\alpha) - \alpha K u^0 + \frac{1}{\alpha}\partial_i(\alpha) u^i + \Gamma^j_{ji} u^i. \tag{5}$$

In the above, we have omitted the ${}^{(4)}$ from the spatial parts of the Christoffel symbols since when $\beta^i = 0$ these are equivalent to those associated with the three-metric γ_{ij}. We have also used the time components of the four-dimensional Christoffel symbols, namely

$$ {}^{(4)}\Gamma^0_{00} = \frac{1}{\alpha}\partial_0 \alpha, \quad {}^{(4)}\Gamma^0_{0i} = \frac{1}{\alpha}\partial_i \alpha, \quad {}^{(4)}\Gamma^0_{ij} = -\frac{1}{\alpha} K_{ij}, \tag{6}$$

$$ {}^{(4)}\Gamma^i_{00} = \alpha \gamma^{ij}\partial_j \alpha, \quad {}^{(4)}\Gamma^i_{0k} = -\alpha K^i{}_k, \tag{7}$$

in deriving the above (and below).

The expansion scalar is currently calculated in `get_expansion_shear_vort` in `roots.f90`.

3.2.2 Shear tensor

The shear tensor is defined as

$$\sigma_{\mu\nu} \equiv b^{\alpha}{}_{\mu} b^{\beta}{}_{\nu} \nabla_{(\alpha} u_{\beta)} - \frac{1}{3}\Theta b_{\mu\nu}, \tag{8}$$

where the projection tensor is

$$b_{\mu\nu} = g_{\mu\nu} + u_{\mu}u_{\nu}, \tag{9}$$

and the round brackets indicate symmetrisation over the contained indices. Expanding the covariant derivative and simplifying gives

$$\begin{aligned}\sigma_{\mu\nu} = b^{0}{}_{\mu} b^{0}{}_{\nu} \left(\partial_0 u_0 - \Gamma^{\alpha}_{00} u_{\alpha}\right) + \frac{1}{2}\left(\partial_0 u_i + \partial_i u_0 - 2\Gamma^{\alpha}_{0i} u_{\alpha}\right)\left(b^{0}{}_{\mu} b^{i}{}_{\nu} + b^{i}{}_{\mu} b^{0}{}_{\nu}\right) \\ + \frac{1}{2} b^{i}{}_{\mu} b^{j}{}_{\nu}\left(\partial_i u_j + \partial_j u_i - 2\Gamma^{\alpha}_{ij} u_{\alpha}\right) - \frac{1}{3}\Theta b_{\mu\nu}.\end{aligned} \tag{10}$$

The rate of shear is then $\sigma^2 \equiv 1/2 \sigma_{\mu\nu}\sigma^{\mu\nu}$. We note that since $b_{\mu\nu}$ is not the three metric describing the simulation hypersurfaces then the components $\sigma_{00}, \sigma_{0i} \neq 0$. These components are

$$\begin{aligned}\sigma_{00} = \left(b^{0}{}_{0}\right)^2 \left(\partial_0 u_0 - \Gamma^{\alpha}_{00} u_{\alpha}\right) + b^{0}{}_{0} b^{i}{}_{0}\left(\partial_0 u_i + \partial_i u_0 - 2\Gamma^{\alpha}_{0i} u_{\alpha}\right) \\ + \frac{1}{2} b^{i}{}_{0} b^{j}{}_{0}\left(\partial_i u_j + \partial_j u_i - 2\Gamma^{\alpha}_{ij} u_{\alpha}\right) - \frac{1}{3}\Theta b_{00},\end{aligned} \tag{11}$$

where

$$b^{0}{}_{0} = 1 - \Gamma^2, \quad b^{i}{}_{0} = -\alpha\Gamma u^i, \quad b_{00} = \alpha^2\left(\Gamma^2 - 1\right), \tag{12}$$

and

$$\begin{aligned}\sigma_{0i} = b^{0}{}_{0} b^{0}{}_{i}\left(\partial_0 u_0 - \Gamma^{\alpha}_{00} u_{\alpha}\right) + \frac{1}{2}\left(b^{0}{}_{0} b^{k}{}_{i} + b^{k}{}_{0} b^{0}{}_{i}\right)\left(\partial_0 u_k + \partial_k u_0 - 2\Gamma^{\alpha}_{0k} u_{\alpha}\right) \\ + \frac{1}{2} b^{k}{}_{0} b^{j}{}_{i}\left(\partial_k u_j + \partial_j u_k - 2\Gamma^{\alpha}_{kj} u_{\alpha}\right) - \frac{1}{3}\Theta b_{0i},\end{aligned} \tag{13}$$

where

$$b^{0}{}_{i} = u^0 u_i, \quad b_{0i} = -\alpha^2 u^0 u_i. \tag{14}$$

The shear tensor (and scalar) is currently calculated in `get_expansion_shear_vort` in `roots.f90`.

3.2.3 Vorticity tensor

The vorticity tensor is defined as

$$\omega_{\mu\nu} \equiv b^{\alpha}{}_{\mu} b^{\beta}{}_{\nu} \nabla_{[\alpha} u_{\beta]} \tag{15}$$

where the square brackets imply anti-symmetrisation across the enclosed indices, i.e.

$$\omega_{\mu\nu} = b^{\alpha}{}_{\mu} b^{\beta}{}_{\nu} \frac{1}{2}\left(\nabla_{\alpha} u_{\beta} - \nabla_{\beta} u_{\alpha}\right), \tag{16}$$

expanding the covariant derivative we find

$$\omega_{\mu\nu} = b^{\alpha}{}_{\mu} b^{\beta}{}_{\nu} \frac{1}{2}\left(\partial_{\alpha} u_{\beta} - \Gamma^{\gamma}_{\alpha\beta} u_{\gamma} - \partial_{\beta} u_{\alpha} + \Gamma^{\gamma}_{\alpha\beta} u_{\gamma}\right), \tag{17}$$

$$= b^{\alpha}{}_{\mu} b^{\beta}{}_{\nu} \frac{1}{2}\left(\partial_{\alpha} u_{\beta} - \partial_{\beta} u_{\alpha}\right), \tag{18}$$

i.e. the symmetric parts (Christoffel symbols) cancel. The vorticity scalar is then $\omega^2 \equiv (1/2)\,\omega_{\mu\nu}\omega^{\mu\nu}$.

The components, as calculated in `mescaline`, are

$$\omega_{0k} = \frac{1}{2}\left(b^{0}{}_{0} b^{i}{}_{k} - b^{i}{}_{0} b^{0}{}_{k}\right)\left(\partial_0 u_i - \partial_i u_0\right) + \frac{1}{2} b^{i}{}_{0} b^{j}{}_{k}\left(\partial_i u_j - \partial_j u_i\right), \tag{19}$$

$$\omega_{kl} = \frac{1}{2}\left(b^{0}{}_{k} b^{i}{}_{l} - b^{i}{}_{k} b^{0}{}_{l}\right)\left(\partial_0 u_i - \partial_i u_0\right) + \frac{1}{2} b^{i}{}_{k} b^{j}{}_{l}\left(\partial_i u_j - \partial_j u_i\right), \tag{20}$$

and

$$\omega_{00} = \frac{1}{2} b^{i}{}_{0} b^{j}{}_{0}\left(\partial_i u_j - \partial_j u_i\right) = 0, \tag{21}$$

which we can see if we expand out the sum over i, j.

The vorticity tensor (and scalar) is currently calculated in `get_expansion_shear_vort` in `roots.f90`.

3.2.4 Four-acceleration

Since we have a strictly non-zero pressure in our simulations, this means we will have a small amount of non-zero four-acceleration. This is defined as

$$a^\mu \equiv u^\nu \nabla_\nu u^\mu, \tag{22}$$

expanding out the covariant derivative and sums we arrive at the following for each component

$$a^0 = u^i \partial_i u^0 - \frac{u^0}{\alpha^2}\partial_0 u_0 - \frac{(u^0)^2}{\alpha}\partial_0 \alpha + \frac{2u^0 u^i}{\alpha}\partial_i \alpha - \frac{1}{\alpha}K_{ij}u^i u^j, \tag{23}$$

$$a^k = u^0 \gamma^{kj}\partial_0 u_j + u^i \partial_i u^k + (u^0)^2 \alpha \gamma^{kj}\partial_j \alpha + u^i u^j \Gamma^k_{ij}. \tag{24}$$

In simplifying the above we have used (4) above and (35) below. The calculation of the four-acceleration is implemented in `get_fluid_curvature` in `roots.f90` due to the fact that a^μ and $\mathcal{R}_f$ share several common terms (see below).

We also calculate the magnitude of the four-acceleration

$$|a| \equiv \sqrt{a^\mu a_\mu}. \tag{25}$$

The four-acceleration is currently calculated in `get_fluid_curvature` in `roots.f90`.

3.2.5 Kinematical backreaction

The kinematical backreaction term $\tilde{Q}_D$ is

$$\tilde{Q}_D \equiv \frac{2}{3}\left(\langle\tilde{\Theta}^2\rangle^b_D - \langle\tilde{\Theta}\rangle^b_D{}^2\right) - 2\langle\tilde{\sigma}^2\rangle^b_D + 2\langle\tilde{\omega}^2\rangle^b_D, \tag{26}$$

with

$$\tilde{\Theta} \equiv \frac{\alpha}{\Gamma}\Theta, \quad \tilde{\sigma}^2 \equiv \left(\frac{\alpha}{\Gamma}\right)^2 \sigma^2, \quad \tilde{\omega}^2 \equiv \left(\frac{\alpha}{\Gamma}\right)^2 \omega^2, \tag{27}$$

see also Buchert et al. (2020). This is currently calculated in `get_backreaction_omegas` in `roots.f90`.

3.2.6 Fluid-restframe 3-curvature

Distinctive from the Ricci 3-curvature of the *spatial slices*, the fluid-restframe 3-curvature (Buchert et al., 2020) is defined as

$$\mathcal{R}_f \equiv \nabla_\mu u^\nu \nabla_\nu u^\mu - \nabla_\mu u^\mu \nabla_\nu u^\nu + {}^{(4)}R + 2R_{\mu\nu}u^\mu u^\nu, \tag{28}$$

$$= \nabla_\mu u^\nu \nabla_\nu u^\mu - \Theta^2 + {}^{(4)}R + 2R_{\mu\nu}u^\mu u^\nu, \tag{29}$$

where $R_{\mu\nu}$ is the four-dimensional Ricci curvature, and ${}^{(4)}R \equiv g^{\mu\nu}R_{\mu\nu}$ is its trace. We find the components of the symmetric $R_{\mu\nu}$, in terms of purely three-dimensional objects, are

$$R_{00} = \alpha\partial_j(\alpha)\partial_i\gamma^{ij} + \alpha\gamma^{ij}\partial_i\partial_j\alpha + \alpha\partial_0 K + \alpha\Gamma^i_{ji}\gamma^{jk}\partial_k\alpha - \alpha^2 K_{ij}K^{ij}, \tag{30}$$

$$R_{0i} = \alpha\left(\partial_i K - \nabla_j K^j{}_i\right) = \alpha\partial_i K - \alpha\partial_j K^j{}_i - \alpha\Gamma^j_{kj}K^k{}_i + \alpha\Gamma^j_{ki}K^k{}_j, \tag{31}$$

$$R_{ij} = {}^{(3)}\mathcal{R}_{ij} + KK_{ij} - 2K^l{}_j K_{il} - \frac{1}{\alpha}\left(\partial_0 K_{ij} + \partial_i\partial_j\alpha - \partial_k(\alpha)\Gamma^k_{ij}\right), \tag{32}$$

and the trace is then

$${}^{(4)}R = -\frac{1}{\alpha^2}R_{00} + \gamma^{ij}R_{ij}. \tag{33}$$

The first term in (29) can be written as (after expanding covariant derivatives, expanding brackets, expanding sums, simplifying, and re-factorising)

$$\begin{aligned}\nabla_\mu u^\nu \nabla_\nu u^\mu = &\left(\frac{u^i}{\alpha}\partial_i\alpha - \frac{1}{\alpha^2}\partial_0 u_0 - \frac{\Gamma}{\alpha^2}\partial_0\alpha\right)^2 \\ &+ 2\left(\partial_0 u^i + \Gamma\gamma^{ij}\partial_j\alpha - \alpha K^i{}_j u^j\right)\left(\partial_i u^0 + \frac{\Gamma}{\alpha^2}\partial_i\alpha - \frac{u^j}{\alpha}K_{ij}\right) \\ &+ \left(\partial_i u^j - \Gamma K^j{}_i + \Gamma^j_{ik}u^k\right)\left(\partial_j u^i - \Gamma K^i{}_j + \Gamma^i_{jl}u^l\right),\end{aligned} \tag{34}$$

where $\partial_0 u^0$ is given by (4), and we can write

$$\partial_0 u^i = \gamma^{ij}\partial_0 u_j + 2\alpha u^j \gamma^{ik} K_{kj}, \tag{35}$$

to minimise calculations, since $\partial_0 u_j$ is already calculated for the shear tensor.

The fluid curvature is currently calculated in `get_fluid_curvature` in `roots.f90`.

3.3 Averaging process

Simulations of nonlinear structure formation in GR (using the BSSN formalism) require a choice of spatial hypersurface to perform the evolution. When structures become nonlinear and stream-crossing occurs, the surface that is everywhere comoving with the fluid flow is no longer well-defined (or no longer exists at all). For this reason, we sometimes need to perform simulation in some arbitrary non-comoving spatial slicing, but when we calculate averages we still want to ensure we are looking at interesting quantities that are related to the fluid itself, and not just the arbitrary coordinates of the slice. The formalism developed in Buchert et al. (2020) presents a fluid-intrinsic averaging procedure that uses arbitrary non-comoving spatial slices. Here we present the details of the averaging itself in `mescaline` using this formalism.

Consider a compact domain $\mathcal{D}$ which exists on the arbitrary non-comoving surface, and is transported in time along the fluid flow lines (note that this transportation in time is not yet properly implemented in `mescaline`, since this would involve also transporting the *edges* of the domain, which is a complicated process. As long as velocities remain small, this should not introduce too much of a correction).

3.3.1 Intrinsic definitions

The intrinsic (or fluid) volume of the domain is

$$V_D^b \equiv \int_D \sqrt{b}\, d^3X, \tag{36}$$

where b is the determinant of the projection tensor (9). The intrinsic (or fluid-volume) average of a scalar ψ is then

$$\langle\psi\rangle_D^b = \frac{\int_D \psi\sqrt{b}\, d^3X}{V_D^b}. \tag{37}$$

We can also define an effective "scale factor" describing the time evolution of the size of the domain, namely,

$$a_D^b(t) \equiv \left(V_D^b(t)\right)^{1/3}, \tag{38}$$

where we note we have left out the normalisation by $V_D^b(t_{\rm init})$ to be consistent with choices of $a_{\rm init} \neq 1$.

3.3.2 Extrinsic definitions

The extrinsic (coordinate) volume is defined as

$$V_D \equiv \int_D \sqrt{h}\, d^3X, \tag{39}$$

where $h_{\mu\nu} = g_{\mu\nu} + n_\mu n_\nu$ is the projection tensor of the non-comoving hypersurface (spatial metric in the case of 3+1 NR). The extrinsic average of ψ is then

$$\langle\psi\rangle_D = \frac{\int_D \psi\sqrt{h}\, d^3X}{V_D}. \tag{40}$$

3.3.3 Relations between the two

The two volume elements can be related by $\sqrt{b}\, d^3X = \Gamma\sqrt{h}\, d^3X$, and we note that both of these are invariant under a change of spatial coordinates (see p.35, footnote 15 in Buchert et al., 2020).

Using this relation, V_D^b is related to the extrinsic volume, V_D, via

$$V_D^b = V_D\langle\Gamma\rangle_D, \tag{41}$$

$$= \int_D \Gamma\sqrt{h}\, d^3X, \tag{42}$$

so the effective scale factor becomes

$$a_D^b = \left(\frac{\int_D \Gamma \sqrt{h}\, d^3X}{\int_D \Gamma \sqrt{h}\, d^3X|_{t_{\rm init}}} \right)^{1/3}. \tag{43}$$

The averages can be related by

$$\langle \psi \rangle_D^b = \frac{\langle \Gamma \psi \rangle_D}{\langle \Gamma \rangle_D}. \tag{44}$$

Here, $\Gamma \equiv 1/\sqrt{1 - v^\alpha v_\alpha}$ is the Lorentz factor, with v^α the peculiar velocity describing the tilt between the fluid-orthogonal frame and the simulation slicing.

3.3.4 Calculating the average within a domain

In the main loop of `ricci.f90` (and `get_expansion_shear_vort` in `roots.f90`) we calculate Θ, σ^2, ω^2, $\mathcal{R}_f$, and Γ at each point in the domain. During this process, we calculate the average by summing over all points and therefore approximating the integral (40).

With the user-chosen radius of the spherical domain $\mathcal{D}$ (or if set to zero, the whole box), and the position of the origin of the sphere, we first check if the current x, y, z position is inside the sphere (by looping over all spheres if necessary). If $r_D \neq 0$, we calculate the distance from the current point to the centre of the sphere

$$r_{\rm temp}^2 = (x - {\rm origin}[1])^2 + (y - {\rm origin}[2])^2 + (z - {\rm origin}[3])^2, \tag{45}$$

and if the condition $r_{\rm temp}^2 \leq r_D^2$ is met, then we add the current value of, say, ψ, to the sum

$${\rm sum}(\psi)_D = {\rm sum}(\psi)_D + \psi(x, y, z) \sqrt{h(x, y, z)}\, \Delta x^3. \tag{46}$$

Here, (x, y, z) represents the current grid cell, and Δx is the size of the grid cell in each dimension (with $\Delta x = \Delta y = \Delta z$, see Section 2 for a full list of assumptions made).

In `get_backreaction_omegas` in `roots.f90` the sums are then normalised using the average Lorentz factor (as per (44), where the volume normalisations cancel out), so that the average becomes

$$\langle \psi \rangle_D^b = \frac{{\rm sum}(\Gamma \psi)_D}{{\rm sum}(\Gamma)_D}. \tag{47}$$

3.3.5 Conversion of averages

The quantities averaged are the "tilde" quantities (27) using the prefactor α/Γ ("threading lapse") introduced in Buchert et al. (2020). In line with (44), we also include an additional factor of Γ in each average to calculate the intrinsic fluid average.

To summarise, the averages calculated are explicitly

$$\langle \tilde{\Theta} \rangle_D^b = {\rm sum}\left(\Gamma \times \frac{\alpha}{\Gamma} \times \Theta \right) \times \frac{1}{{\rm sum}(\Gamma)}, \tag{48}$$

$$\langle \tilde{\Theta}^2 \rangle_D^b = {\rm sum}\left(\Gamma \times \left(\frac{\alpha}{\Gamma} \right)^2 \times \Theta^2 \right) \times \frac{1}{{\rm sum}(\Gamma)}, \tag{49}$$

$$\langle \tilde{\sigma}^2 \rangle_D^b = {\rm sum}\left(\Gamma \times \left(\frac{\alpha}{\Gamma} \right)^2 \times \sigma^2 \right) \times \frac{1}{{\rm sum}(\Gamma)}, \tag{50}$$

$$\langle \tilde{\omega}^2 \rangle_D^b = {\rm sum}\left(\Gamma \times \left(\frac{\alpha}{\Gamma} \right)^2 \times \omega^2 \right) \times \frac{1}{{\rm sum}(\Gamma)}, \tag{51}$$

$$\langle \tilde{\mathcal{R}}_f \rangle_D^b = {\rm sum}\left(\Gamma \times \left(\frac{\alpha}{\Gamma} \right)^2 \times \mathcal{R}_f \right) \times \frac{1}{{\rm sum}(\Gamma)}, \tag{52}$$

$$\langle \tilde{\rho} \rangle_D^b = {\rm sum}\left(\Gamma \times \left(\frac{\alpha}{\Gamma} \right)^2 \times \rho \right) \times \frac{1}{{\rm sum}(\Gamma)}, \tag{53}$$

$$\tag{54}$$

where we have intentionally not cancelled factors for clarity. We use the above averages to calculate the kinematical backreaction (26).

3.3.6 Cosmological parameters

Once we have calculated the averages (48) and the kinematical backreaction, we calculate the cosmological parameters. Still in `get_backreaction_omegas` in `roots.f90`, the Hubble parameter is

$$\mathcal{H}_D = \frac{1}{3}\langle\tilde{\Theta}\rangle^b_D, \tag{55}$$

and the cosmological parameters are then

$$\Omega_m = \frac{8\pi\langle\tilde{\rho}\rangle^b_D}{3\mathcal{H}^2_D}, \tag{56}$$

$$\Omega_R = -\frac{\langle\tilde{\mathcal{R}}_f\rangle^b_D}{6\mathcal{H}^2_D}, \tag{57}$$

$$\Omega_Q = -\frac{\tilde{Q}_D}{6\mathcal{H}^2_D}, \tag{58}$$

3.4 Weyl tensor

We may be interested in calculating the Weyl tensor from a simulation, specifically it's electric or magnetic part on the grid. We have implemented the calculation of the electric part of the Weyl tensor in the fluid frame, defined as

$$E_{\alpha\beta} \equiv C_{\mu\nu\lambda\rho}u^\mu h^\nu{}_\alpha u^\lambda h^\rho{}_\beta, \tag{59}$$

where u^μ is the 4–velocity of the fluid, and the projection tensor $h^\mu{}_\alpha \equiv g^\mu{}_\alpha + u^\mu u_\alpha = \delta^\mu{}_\alpha + u^\mu u_\alpha$ was introduced in earlier sections. The Weyl tensor itself is defined as the traceless part of the Riemann tensor, namely

$$C_{\mu\nu\lambda\rho} \equiv R_{\mu\nu\lambda\rho} + \left(g_{\mu[\lambda}R_{\rho]\nu} - g_{\nu[\lambda}R_{\rho]\mu}\right) + \frac{1}{3}R\, g_{\mu[\lambda}g_{\rho]\nu} \tag{60}$$

for which we can expand the antisymmetric parts to get

$$C_{\mu\nu\lambda\rho} \equiv R_{\mu\nu\lambda\rho} + \frac{1}{2}\Big[g_{\mu\rho}R_{\nu\lambda} + g_{\nu\lambda}R_{\mu\rho} - g_{\mu\lambda}R_{\nu\rho} - g_{\nu\rho}R_{\mu\lambda}\Big] + \frac{1}{6}\Big(g_{\mu\lambda}g_{\nu\rho} - g_{\mu\rho}g_{\nu\lambda}\Big)R, \tag{61}$$

where $R_{\mu\nu\lambda\rho}$ is the Riemann tensor, $R_{\mu\nu}$ is the 4-Ricci tensor and $R \equiv g^{\mu\nu}R_{\mu\nu}$ is its trace. To calculate $E_{\alpha\beta}$ we implement a similar method to calculating the Weyl tensor as is done in the raytracer in `mescaline` (see `doc/Raytracing_mescaline.tex`). This is motivated by the methods outlined in Appendix B of Giblin et al. (2016), and involves simplifying the indices of the Weyl tensor in the following way:

$$C_{AB} = C_{\mu\nu\lambda\rho}, \tag{62}$$

such that A, B take values over all combinations of $\mu\nu, \lambda\rho$, respectively. C_{AB} is symmetric because of the symmetries of the Riemann tensor in $R_{\mu\nu\lambda\rho} = R_{\lambda\rho\mu\nu}$, and it is antisymmetric in changes $A = \mu\nu \to \nu\mu$ since $R_{\mu\nu\lambda\rho} = -R_{\nu\mu\lambda\rho}$. These symmetries, and the fact that Weyl components with A as repeated indices $\mu = \nu$, i.e. $A = 00, 11, \ldots$ are zero allow us to simplify the calculation of the electric Weyl significantly. To do so we also define

$$F^A_\alpha \equiv u^\mu h^\nu{}_\alpha, \tag{63}$$

such that (59) becomes

$$E_{\alpha\beta} = C_{AB}F^A_\alpha F^B_\beta, \tag{64}$$

where $A, B \in [01, 02, 03, 12, 13, 23, 10, 20, 30, 21, 31, 32]$. We can simplify this further by denoting "forward" and "backward" indices $A_f \in [01, 02, 03, 12, 13, 23]$ and $A_b \in [10, 20, 30, 21, 31, 32]$, and expanding the above sum, i.e.,

$$E_{\alpha\beta} = C_{A_fB}F^{A_f}_\alpha F^B_\beta + C_{A_bB}F^{A_b}_\alpha F^B_\beta, \tag{65}$$

exploiting the antisymmetry of C_{AB} in $A_f \to A_b$ allows us to eventually write (after similarly expanding the index $B = [B_f, B_b]$)

$$E_{\alpha\beta} = 4C_{AB}F^{[A]}_\alpha F^{[B]}_\beta, \tag{66}$$

where we have now removed the subscript 'f' for brevity, and have $A, B \in [01, 02, 03, 12, 13, 23]$ for the above expression. The anti-symmetric part of F_α^A is defined as

$$F_\alpha^{[A]} \equiv \frac{1}{2}\left(F_\alpha^{A_f} - F_\alpha^{A_b}\right). \tag{67}$$

We now need only calculate the symmetric $(6, 6)$ tensor C_{AB}, for which we utilise the already existing routine `get_RAB` in `riemann.f90` to calculate the Riemann tensor using the same index simplification, and we therefore need only calculate the terms in parenthesis in (60). These terms are simple provided we already calculate $R_{\mu\nu}$ and R in `ricci.f90`.

The calculation of the electric part of the Weyl tensor is implemented in `get_electric_weyl` in `weyl.f90` (and other routines within), and is intended to be called from the main spatial loop of `ricci.f90`.

References

Buchert, T., Mourier, P., and Roy, X. (2018). Cosmological backreaction and its dependence on spacetime foliation. *Classical and Quantum Gravity*, 35(24):24LT02.

Buchert, T., Mourier, P., and Roy, X. (2020). On average properties of inhomogeneous fluids in general relativity III: general fluid cosmologies. *General Relativity and Gravitation*, 52(3):27.

Giblin, Jr., J. T., Mertens, J. B., and Starkman, G. D. (2016). Observable Deviations from Homogeneity in an Inhomogeneous Universe. ApJ, 833:247.

Macpherson, H. J. (2019). Inhomogeneous cosmology in an anisotropic Universe. *arXiv e-prints*, page arXiv:1910.13380.

Macpherson, H. J., Lasky, P. D., and Price, D. J. (2018). The Trouble with Hubble: Local versus Global Expansion Rates in Inhomogeneous Cosmological Simulations with Numerical Relativity. ApJ, 865:L4.

Macpherson, H. J., Price, D. J., and Lasky, P. D. (2019). Einstein's Universe: Cosmological structure formation in numerical relativity. Phys. Rev. D, 99(6):063522.